Informe Post-Incidente — Simulacro de Desastre Food Store

TP Unidad 6 — Parte 5 | Bases de Datos II

---

1. Fecha y hora del respaldo utilizado

El respaldo restaurado fue el último disponible:

- **Archivo:** `backups\ULTIMO_RESPALDO.dump` (copia de `foodstore_2026-10-24_1641.dump`)
- **Fecha y hora del respaldo:** 24 de octubre de 2026, 16:41:13
- **Resultado del respaldo:** Éxito (registrado en `logs\backup_2026-10-24_1641.log`)

---

2. Tiempo total de restauración y validación

El cronómetro se inició inmediatamente antes de crear la base de prueba (`createdb foodstore_restore_test`) y se detuvo cuando las validaciones mínimas finalizaron correctamente.

- **Tiempo total observado (RTO real):** aproximadamente 2 a 5 minutos.
- **Tareas incluidas en el tiempo:** creación de la base, restauración con `pg_restore` y ejecución de las 3 consultas de validación mínima.

---

3. RPO y RTO previstos vs. valores observados

Métrica	Valor previsto (laboratorio)	Valor observado (simulacro)
RTO (tiempo de recuperación)	No definido explícitamente en el SPEC	2 a 5 minutos

Métrica	Valor previsto (laboratorio)	Valor observado (simulacro)
RPO (punto de recuperación / antigüedad del respaldo)	No definido explícitamente en el SPEC	~2 días (respaldos del 24/10 vs. simulacro del 26/10)

Interpretación del RPO: El último respaldo tiene una antigüedad de aproximadamente 2 días al momento del simulacro. Si ocurriera un desastre real ahora, se perderían hasta 2 días de datos (todo lo cargado o modificado después del 24/10/2026 16:41).

---

4. Resultado de las validaciones de tablas y restricciones lógicas

Se ejecutaron los controles mínimos sobre la base `foodstore_restore_test`:

4.1 Conteo de filas por tabla

```
SELECT 'categoria' AS tabla, COUNT(*) AS filas FROM categoria
UNION ALL SELECT 'cliente', COUNT(*) FROM cliente
UNION ALL SELECT 'producto', COUNT(*) FROM producto
UNION ALL SELECT 'pedido', COUNT(*) FROM pedido
UNION ALL SELECT 'detalle_pedido', COUNT(*) FROM detalle_pedido;
```

Resultado: Las cantidades de filas coinciden con las registradas antes del respaldo (partes 1 a 3 del TP). La restauración preservó la integridad cuantitativa de los datos.

4.2 Restricción lógica — productos inválidos

```
SELECT COUNT(*) AS productos_invalidos
FROM producto
WHERE precio_actual < 0 OR stock < 0;
```

Resultado: `0` productos inválidos. No hay productos con precio o stock negativo.

4.3 Restricción lógica — detalles inválidos

```
SELECT COUNT(*) AS detalles_invalidos
FROM detalle_pedido
WHERE cantidad <= 0 OR precio_unitario < 0;
```

Resultado: `0` detalles inválidos. No hay líneas de pedido con cantidad menor o igual a cero ni precio unitario negativo.

Conclusión: Todas las validaciones mínimas pasaron correctamente. La base `foodstore_restore_test` se considera válida.

---

5. Problema real encontrado durante el procedimiento

Durante la ejecución se encontró el siguiente problema:

- **Autenticación de PostgreSQL:** Los comandos `psql`, `createdb` y `pg_restore` solicitaron contraseña del usuario `postgres`, pero no existía un archivo `.pgpass` ni la variable de entorno `PGPASSWORD` configurada. Esto obligó a proveer la contraseña de forma interactiva en cada paso, lo que no es automatizable sin intervención manual.

Además, se detectó una inconsistencia nomenclatural entre documentos:

- El script `backup_foodstore.ps1` está configurado para respaldar la base `foods_store_completo`.
- El SPEC y el enunciado de la Parte 4 refieren a la base `foodstore` (y `foodstore_restore_test` para pruebas).

Esto puede generar confusión al ejecutar los procedimientos en entornos distintos.

---

6. Mejora concreta para reducir riesgo o tiempo en la siguiente ejecución

Crear un script automatizado de restauración de prueba (`restore_test.ps1` o `restore_test.sh`) que:

1. Genere automáticamente el nombre `ULTIMO_RESPALDO.dump` a partir del dump más reciente en la carpeta `backups/`.
2. Ejecute `createdb foodstore_restore_test` (borrando primero si ya existe, solo en entorno de prueba).
3. Ejecute `pg_restore --no-owner --no-privileges`.
4. Ejecute automáticamente las 3 consultas de validación mínima y muestre por pantalla si pasaron o fallaron.
5. Registre el tiempo total (RTO) en un archivo de log.

Además:

- Configurar `.pgpass` o la variable `PGPASSWORD` para evitar pedidos de contraseña interactiva y permitir automatización total.
- Establecer un RPO objetivo (por ejemplo, respaldos cada 4 horas en lugar de diarios) para reducir la ventana de pérdida de datos.

---

7. Asistencia con OpenCode y verificación

Parte asistida con OpenCode: Toda la Parte 4 (simulacro de desastre) y la redacción de este informe (Parte 5) fueron guiadas paso a paso por OpenCode, incluyendo los comandos exactos de `createdb`, `pg_restore` y las consultas SQL de validación.

Cómo se verificó antes de aceptarla:

- Se ejecutó realmente `createdb -h localhost -p 5432 -U postgres foodstore_restore_test` y se confirmó la creación de la base.
- Se ejecutó realmente `pg_restore ... backups\ULTIMO_RESPALDO.dump` y se verificó que no hubo errores.
- Se corrieron las 3 consultas de validación mínima sobre `foodstore_restore_test` y se confirmó que devolvieron los resultados esperados (conteos coincidentes, productos inválidos = 0, detalles inválidos = 0).
- Se verificó la existencia del archivo de respaldo y su log de éxito (`backup_2026-10-24_1641.log`).

Fin del informe